

Practical Week-9

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int source, destination;
    char buffer[1024];
    ssize_t bytes;
    off_t size;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);
    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    // Get source file size using lseek
    size = lseek(source, 0, SEEK_END);
    printf("Source file size: %ld bytes\n", (long)size);
    lseek(source, 0, SEEK_SET); // Reset file offset back to start

    // Copy file content
    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + ▾  
imvarls@LAPTOP-VQ4D70L7:~$ nano copy_syscalls.c  
imvarls@LAPTOP-VQ4D70L7:~$ gcc copy_syscalls.c -o copy_syscalls  
imvarls@LAPTOP-VQ4D70L7:~$ ./copy_syscalls  
Usage: ./copy_syscalls source destination  
imvarls@LAPTOP-VQ4D70L7:~$ echo "Hello Arun! thia is a file copy test." > input.txt  
imvarls@LAPTOP-VQ4D70L7:~$ echo "Hello Arun! this is a file copy test." > input.txt  
imvarls@LAPTOP-VQ4D70L7:~$ cat input.txt  
Hello Arun! this is a file copy test.  
imvarls@LAPTOP-VQ4D70L7:~$ gcc copy_syscalls.c -o copy_syscalls  
imvarls@LAPTOP-VQ4D70L7:~$ ./copy_syscalls  
Usage: ./copy_syscalls source destination  
imvarls@LAPTOP-VQ4D70L7:~$ ./copy_syscalls input.txt copy.txt  
Source file size: 38 bytes  
File copied successfully using system calls.  
imvarls@LAPTOP-VQ4D70L7:~$ cat copy.txt  
Hello Arun! this is a file copy test.  
imvarls@LAPTOP-VQ4D70L7:~$
```

Modified program to use lseek():

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int source, destination;
    char buffer[1024];
    ssize_t bytes;
    off_t size;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);
    if (source < 0) {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (destination < 0) {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    // Get source file size using lseek
    size = lseek(source, 0, SEEK_END);
    printf("Source file size: %ld bytes\n", (long)size);
    lseek(source, 0, SEEK_SET); // Reset file offset back to start

    // Copy file content
    while ((bytes = read(source, buffer, sizeof(buffer))) > 0) {
        write(destination, buffer, bytes);
    }

    close(source);
    close(destination);

    printf("File copied successfully using system calls.\n");
    return 0;
}
```

1B



Ubuntu



GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    FILE *source, *destination;
    char buffer[1024];
    size_t bytes;

    if (argc != 3) {
        printf("Usage: %s source destination\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");
    if (source == NULL) {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");
    if (destination == NULL) {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytes = fread(buffer, 1, sizeof(buffer), source)) > 0) {
        fwrite(buffer, 1, bytes, destination);
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");
    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7:~$ nano copy_stdio.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc copy_stdio.c -o copy_stdio
imvarls@LAPTOP-VQ4D70L7:~$ ./copy_stdio input.txt copy2.txt
File copied successfully using standard library functions.
imvarls@LAPTOP-VQ4D70L7:~$ cat copy2.txt
Hello Arun! this is a file copy test.
imvarls@LAPTOP-VQ4D70L7:~$
```

2. Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
imvarls@LAPTOP-VQ4D70L7: GNU nano 8.7.1
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(void) {
    int input_fd, output_fd;

    input_fd = open("input.txt", O_RDONLY);
    if (input_fd < 0) {
        perror("input.txt");
        return 1;
    }

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd < 0) {
        perror("output.txt");
        close(input_fd);
        return 1;
    }

    // Redirect standard input and standard output
    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    close(input_fd);
    close(output_fd);

    char buffer[100];
    printf("This output is redirected to output.txt\n");
    printf("Reading from redirected input:\n");

    while (read(STDIN_FILENO, buffer, sizeof(buffer) - 1) > 0) {
        printf("%s", buffer);
    }

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + ∨  
imvarls@LAPTOP-VQ4D70L7:~$ nano redirection.c  
imvarls@LAPTOP-VQ4D70L7:~$ gcc redirection.c -o redirection  
imvarls@LAPTOP-VQ4D70L7:~$ ./redirection  
imvarls@LAPTOP-VQ4D70L7:~$ cat out.txt  
cat: out.txt: No such file or directory  
imvarls@LAPTOP-VQ4D70L7:~$ cat output.txt  
This output is redirected to output.txt
```